

```
-- =====  
-- ECOMMERCE STORED FUNCTIONS LAB  
-- MySQL Beginner Level  
-- =====
```

```
CREATE DATABASE IF NOT EXISTS ecommerce_db;  
USE ecommerce_db;
```

```
-- =====  
-- DROP OLD TABLES  
-- =====
```

```
DROP TABLE IF EXISTS order_items;  
DROP TABLE IF EXISTS orders;  
DROP TABLE IF EXISTS products;  
DROP TABLE IF EXISTS customers;
```

```
-- =====  
-- CREATE TABLES  
-- =====
```

```
CREATE TABLE customers (  
    customer_id INT AUTO_INCREMENT PRIMARY KEY,  
    customer_name VARCHAR(100),  
    email VARCHAR(100),  
    membership_level VARCHAR(30)  
);
```

```
CREATE TABLE products (  
    product_id INT AUTO_INCREMENT PRIMARY KEY,  
    product_name VARCHAR(100),  
    category VARCHAR(50),  
    unit_price DECIMAL(10,2),  
    stock_qty INT  
);
```

```
CREATE TABLE orders (  
    order_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
customer_id INT,  
order_date DATE,  
order_status VARCHAR(30),  
FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
);
```

```
CREATE TABLE order_items (  
order_item_id INT AUTO_INCREMENT PRIMARY KEY,  
order_id INT,  
product_id INT,  
quantity INT,  
unit_price DECIMAL(10,2),  
FOREIGN KEY (order_id) REFERENCES orders(order_id),  
FOREIGN KEY (product_id) REFERENCES products(product_id)  
);
```

```
-- =====  
-- POPULATE SAMPLE DATA  
-- =====
```

```
INSERT INTO customers (customer_name, email, membership_level)  
VALUES  
( 'Aung Aung', 'aung@example.com', 'Gold'),  
( 'Su Su', 'su@example.com', 'Silver'),  
( 'Kyaw Kyaw', 'kyaw@example.com', 'Normal');
```

```
INSERT INTO products (product_name, category, unit_price, stock_qty)  
VALUES  
( 'Laptop', 'Electronics', 1200000, 10),  
( 'Mouse', 'Electronics', 25000, 100),  
( 'Keyboard', 'Electronics', 45000, 50),  
( 'Office Chair', 'Furniture', 180000, 20),  
( 'Desk', 'Furniture', 250000, 15);
```

```
INSERT INTO orders (customer_id, order_date, order_status)  
VALUES  
(1, '2026-05-01', 'Completed'),  
(2, '2026-05-02', 'Completed'),
```

```
(3, '2026-05-03', 'Pending');
```

```
INSERT INTO order_items (order_id, product_id, quantity, unit_price)
VALUES
(1, 1, 1, 1200000),
(1, 2, 2, 25000),
(2, 4, 1, 180000),
(2, 3, 1, 45000),
(3, 5, 1, 250000);
```

```
-- =====
-- FUNCTION 1: Calculate order subtotal
-- =====
```

```
DELIMITER $$
```

```
CREATE FUNCTION fn_order_subtotal(p_order_id INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE v_subtotal DECIMAL(10,2);

    SELECT SUM(quantity * unit_price)
    INTO v_subtotal
    FROM order_items
    WHERE order_id = p_order_id;

    RETURN IFNULL(v_subtotal, 0);
END $$
```

```
DELIMITER ;
```

```
-- Test
```

```
110 SELECT fn_order_subtotal(1) AS order_subtotal;
```

Result Grid		 Filter Rows:	Export: 	Wrap Cell Content: 
	order_subtotal			
	▶ 1250000.00			

```
-- =====  
-- FUNCTION 2: Calculate tax amount  
-- =====
```

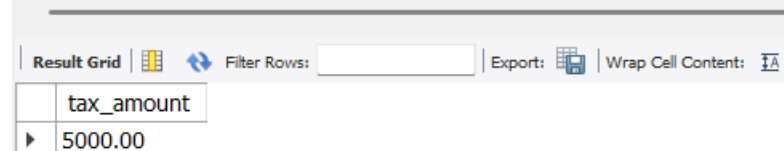
DELIMITER \$\$

```
CREATE FUNCTION fn_tax_amount(p_amount DECIMAL(10,2))  
RETURNS DECIMAL(10,2)  
DETERMINISTIC  
BEGIN  
    RETURN p_amount * 0.05;  
END $$
```

DELIMITER ;

```
-- Test
```

```
128  SELECT fn_tax_amount(100000) AS tax_amount;
```



The screenshot shows a SQL query execution result. The query is `SELECT fn_tax_amount(100000) AS tax_amount;`. The result is displayed in a table with one column named `tax_amount` and one row containing the value `5000.00`. The interface includes a toolbar with options like 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'.

tax_amount
5000.00

```
-- =====  
-- FUNCTION 3: Calculate discount by membership  
-- =====
```

DELIMITER \$\$

```
CREATE FUNCTION fn_membership_discount(  
    p_amount DECIMAL(10,2),  
    p_membership_level VARCHAR(30)  
)  
RETURNS DECIMAL(10,2)  
DETERMINISTIC  
BEGIN  
    DECLARE v_discount DECIMAL(10,2);  
  
    IF p_membership_level = 'Gold' THEN
```

```

        SET v_discount = p_amount * 0.10;
    ELSEIF p_membership_level = 'Silver' THEN
        SET v_discount = p_amount * 0.05;
    ELSE
        SET v_discount = 0;
    END IF;

```

```

    RETURN v_discount;
END $$

```

```

DELIMITER ;

```

```

-- Test

```

```

159      SELECT fn_membership_discount(100000, 'Gold') AS discount_amount;

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	discount_amount			
▶	10000.00			

```

-- =====
-- FUNCTION 4: Calculate final order total
-- subtotal + tax - discount
-- =====

```

```

DELIMITER $$

```

```

CREATE FUNCTION fn_order_final_total(p_order_id INT)
RETURNS DECIMAL(10,2)
READS SQL DATA
BEGIN
    DECLARE v_subtotal DECIMAL(10,2);
    DECLARE v_tax DECIMAL(10,2);
    DECLARE v_discount DECIMAL(10,2);
    DECLARE v_membership VARCHAR(30);
    DECLARE v_final_total DECIMAL(10,2);

    SELECT c.membership_level
    INTO v_membership

```

```

FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.order_id = p_order_id;

SET v_subtotal = fn_order_subtotal(p_order_id);
SET v_tax = fn_tax_amount(v_subtotal);
SET v_discount = fn_membership_discount(v_subtotal, v_membership);

SET v_final_total = v_subtotal + v_tax - v_discount;

RETURN IFNULL(v_final_total, 0);
END $$

DELIMITER ;

```

-- Test

```
196 SELECT fn_order_final_total(1) AS final_order_total;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	final_order_total			
▶	1187500.00			

```

-- =====
-- FUNCTION 5: Check stock status
-- =====

```

DELIMITER \$\$

```

CREATE FUNCTION fn_stock_status(p_stock_qty INT)
RETURNS VARCHAR(30)
DETERMINISTIC
BEGIN
    RETURN CASE
        WHEN p_stock_qty = 0 THEN 'Out of Stock'
        WHEN p_stock_qty < 10 THEN 'Low Stock'
        ELSE 'Available'
    END;
END $$

```

DELIMITER ;

-- Test

```
218  SELECT
219      product_name,
220      stock_qty,
221      fn_stock_status(stock_qty) AS stock_status
222  FROM products;
```

Result Grid			
Filter Rows:			
Export:  Wrap Cell Content: 			
	product_name	stock_qty	stock_status
▶	Laptop	10	Available
	Mouse	100	Available
	Keyboard	50	Available
	Office Chair	20	Available
	Desk	15	Available

-- =====

-- FUNCTION 6: Count customer orders

-- =====

DELIMITER \$\$

```
CREATE FUNCTION fn_customer_order_count(p_customer_id INT)
RETURNS INT
READS SQL DATA
BEGIN
    DECLARE v_total_orders INT;

    SELECT COUNT(*)
    INTO v_total_orders
    FROM orders
    WHERE customer_id = p_customer_id;

    RETURN v_total_orders;
END $$
```

DELIMITER ;

-- Test

```
247  SELECT
248      customer_name,
249      fn_customer_order_count(customer_id) AS total_orders
250  FROM customers;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	customer_name	total_orders			
▶	Aung Aung	1			
	Su Su	1			
	Kyaw Kyaw	1			

```
-- =====
-- FUNCTION 7: Get customer membership label
-- =====
```

DELIMITER \$\$

```
CREATE FUNCTION fn_membership_label(p_membership_level VARCHAR(30))
RETURNS VARCHAR(100)
DETERMINISTIC
BEGIN
    RETURN CASE
        WHEN p_membership_level = 'Gold' THEN 'Gold Member - 10% Discount'
        WHEN p_membership_level = 'Silver' THEN 'Silver Member - 5% Discount'
        ELSE 'Normal Member - No Discount'
    END;
END $$
```

DELIMITER ;

-- Test

```
272  SELECT
273      customer_name,
274      membership_level,
275      fn_membership_label(membership_level) AS membership_description
276  FROM customers;
277
```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 			
	customer_name	membership_level	membership_description
▶	Aung Aung	Gold	Gold Member - 10% Discount
	Su Su	Silver	Silver Member - 5% Discount
	Kyaw Kyaw	Normal	Normal Member - No Discount

-- =====

-- FUNCTION 8: Calculate shipping fee

-- =====

DELIMITER \$\$

CREATE FUNCTION fn_shipping_fee(p_amount DECIMAL(10,2))

RETURNS DECIMAL(10,2)

DETERMINISTIC

BEGIN

IF p_amount >= 500000 THEN

RETURN 0;

ELSE

RETURN 5000;

END IF;

END \$\$

DELIMITER ;

-- Test

```
298  SELECT fn_shipping_fee(300000) AS shipping_fee;
299
```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 	
	shipping_fee
▶	5000.00

```
-- =====  
-- FUNCTION 9: Calculate grand total  
-- final total + shipping fee  
-- =====
```

DELIMITER \$\$

```
CREATE FUNCTION fn_order_grand_total(p_order_id INT)  
RETURNS DECIMAL(10,2)  
READS SQL DATA  
BEGIN  
    DECLARE v_final_total DECIMAL(10,2);  
    DECLARE v_shipping_fee DECIMAL(10,2);  
  
    SET v_final_total = fn_order_final_total(p_order_id);  
    SET v_shipping_fee = fn_shipping_fee(v_final_total);  
  
    RETURN v_final_total + v_shipping_fee;  
END $$
```

DELIMITER ;

-- Test

```
323  SELECT fn_order_grand_total(1) AS grand_total;  
324
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	grand_total			
▶	1187500.00			

-- =====

-- PRACTICAL REPORT QUERY

-- =====

329 • **SELECT**

```
330     o.order_id,
331     c.customer_name,
332     c.membership_level,
333     fn_order_subtotal(o.order_id) AS subtotal,
334     fn_tax_amount(fn_order_subtotal(o.order_id)) AS tax_amount,
335     fn_membership_discount(
336         fn_order_subtotal(o.order_id),
337         c.membership_level
338     ) AS discount_amount,
339     fn_order_final_total(o.order_id) AS final_total,
340     fn_shipping_fee(fn_order_final_total(o.order_id)) AS shipping_fee,
341     fn_order_grand_total(o.order_id) AS grand_total,
342     o.order_status
343 FROM orders o
344 JOIN customers c ON o.customer_id = c.customer_id;
```

Result Grid										
Filter Rows:										
Export: Wrap Cell Content:										
	order_id	customer_name	membership_level	subtotal	tax_amount	discount_amount	final_total	shipping_fee	grand_total	order_status
▶	1	Aung Aung	Gold	1250000.00	62500.00	125000.00	1187500.00	0.00	1187500.00	Completed
	2	Su Su	Silver	225000.00	11250.00	11250.00	225000.00	5000.00	230000.00	Completed
	3	Kyaw Kyaw	Normal	250000.00	12500.00	0.00	262500.00	5000.00	267500.00	Pending

-- =====

-- SHOW ALL FUNCTIONS

-- =====

350 • **SHOW FUNCTION STATUS**

```
351 WHERE Db = 'ecommerce_db';
```

Result Grid						
Filter Rows:						
Export: Wrap Cell Content:						
	Db	Name	Type	Definer	Modified	Created
▶	ecommerce_db	fn_customer_order_count	FUNCTION	root@localhost	2026-05-20 15:52:09	2026-05-20 15:52:09
	ecommerce_db	fn_membership_discount	FUNCTION	root@localhost	2026-05-20 15:38:45	2026-05-20 15:38:45
	ecommerce_db	fn_membership_label	FUNCTION	root@localhost	2026-05-20 15:53:35	2026-05-20 15:53:35
	ecommerce_db	fn_order_final_total	FUNCTION	root@localhost	2026-05-20 15:48:32	2026-05-20 15:48:32
	ecommerce_db	fn_order_grand_total	FUNCTION	root@localhost	2026-05-20 15:55:51	2026-05-20 15:55:51
	ecommerce_db	fn_order_subtotal	FUNCTION	root@localhost	2026-05-20 15:36:54	2026-05-20 15:36:54
	ecommerce_db	fn_shipping_fee	FUNCTION	root@localhost	2026-05-20 15:54:54	2026-05-20 15:54:54
	ecommerce_db	fn_stock_status	FUNCTION	root@localhost	2026-05-20 15:51:19	2026-05-20 15:51:19
	ecommerce_db	fn_tax_amount	FUNCTION	root@localhost	2026-05-20 15:38:06	2026-05-20 15:38:06

```
-- =====  
-- DROP FUNCTION EXAMPLES  
-- =====
```

```
-- DROP FUNCTION IF EXISTS fn_order_subtotal;  
-- DROP FUNCTION IF EXISTS fn_tax_amount;  
-- DROP FUNCTION IF EXISTS fn_membership_discount;  
-- DROP FUNCTION IF EXISTS fn_order_final_total;  
-- DROP FUNCTION IF EXISTS fn_stock_status;  
-- DROP FUNCTION IF EXISTS fn_customer_order_count;  
-- DROP FUNCTION IF EXISTS fn_membership_label;  
-- DROP FUNCTION IF EXISTS fn_shipping_fee;  
-- DROP FUNCTION IF EXISTS fn_order_grand_total;
```